

TrustHome PH — Firebase Setup Plan

A one-time setup, meant to be done together (screen-share/call) so everyone ends up with working access at the same time — this is the literal first task of Week 1, and everything else blocks on it.

Stack note (updated): Firebase Storage now requires a linked credit card (Google changed this in Feb 2026 — it's not something anyone did wrong). Firestore and Authentication are unaffected and still fully free with no card needed. So this plan uses **Firebase for Firestore + Auth, and Cloudinary for file storage** (listing photos, ID/ownership documents) instead — Cloudinary's free tier needs no card at all. Everything else in the System Build Plan (data model, booking logic, trust score, dispute trail) is unchanged.

Who does this: one person (suggest Andresa, since Auth is the first module built) creates the project and adds everyone else. Budget **30-45 minutes** for the whole team to be up and running.

Step 1 — Create the Firebase Project

1. Go to console.firebase.google.com and sign in with a Google account (use one the whole team can access, or plan to add teammates as project members in Step 2).
2. Click **Add project**, name it `trusthome-ph` (or similar — this becomes part of some URLs, so keep it short and no spaces).
3. Disable Google Analytics for this project (not needed, and skips a setup step) → **Create project**.

Step 2 — Add Your Team as Project Members

1. In the Firebase Console, click the **gear icon** → **Users and permissions**.
2. Click **Add member**, enter each teammate's Google account email, set role to **Editor** (Owner only for whoever set it up).
3. Everyone should now see the project in their own Firebase Console.

Step 3 — Enable Firestore

1. In the left sidebar, click **Build** → **Firestore Database**.
2. Click **Create database**.
3. Choose **Start in test mode** for now (this allows open read/write while you're building — **you must lock this down before your defense**, see Step 11).
4. Pick a location close to the Philippines — `asia-southeast1` (Singapore) is the standard choice.

Step 4 — Enable Authentication

1. **Build** → **Authentication** → **Get started**.
2. Under **Sign-in method**, enable **Email/Password**.
3. (Optional, nice for a smoother demo login) also enable **Google** sign-in.

Step 5 — Set Up Cloudinary (for file storage instead of Firebase Storage)

1. Go to cloudinary.com → **Sign up free** — no credit card required.
2. On your Cloudinary dashboard, note your **Cloud Name** (shown right at the top) — you'll need this in Step 8.
3. Go to **Settings** → **Upload** → scroll to **Upload presets** → **Add upload preset**.
4. Set **Signing Mode** to **Unsigned** (this lets the React app upload directly from the browser without needing a backend to sign each request — much simpler for this project's scope).
5. Name the preset `trusthome_uploads`, save it.

Known limitation, worth stating plainly if asked: an unsigned preset means anyone with a file's exact URL can view it — it's not linked or listed anywhere public, but it isn't cryptographically private either. For an ID/ownership document, that's an acceptable tradeoff for an MVP's scope (the same "state the limitation honestly" approach used elsewhere in the proposal), but it's not a production-grade privacy guarantee. If a panelist asks, the honest answer is: *"Uploaded documents are only ever linked from the admin review queue — not publicly listed — but we're not treating this as cryptographically secure storage, the same way we're upfront that document review isn't a legal title search."*

Step 6 — Get the Web App Config (for React)

1. In Project Settings (gear icon) → scroll to **Your apps** → click the `</>` (Web) icon.
2. Name it `trusthome-web` → **Register app**.
3. Firebase shows you a `firebaseConfig` object — copy it, you'll need it in Step 8.
4. This config is **safe to include in your frontend code** — it's a public client identifier, not a secret (Firestore

Security Rules are what actually protect your data, not hiding this config).

Step 7 — Get the Admin Credential (for the Python scraper)

1. Project Settings → **Service Accounts** tab.
2. Click **Generate new private key** → downloads a JSON file.
3. This file is a **full admin credential** — treat it like a password. Rename it to `serviceAccountKey.json` and place it inside the `bank_scraper` project folder (which already has a `.gitignore` entry for it — never commit this file to GitHub).
4. Share this file with Solpico (owns the scraper module) through a private channel (not a public Slack/Discord, not committed to git) — e.g. a DM or a shared private drive folder.

Step 8 — Wire Up the React App

In your React project folder:

```
npm install firebase
```

Create `src/firebase.js`:

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  // paste the config object from Step 6 here
};

const app = initializeApp(firebaseConfig);

export const db = getFirestore(app);
export const auth = getAuth(app);
```

Every teammate building a React feature imports from this one file (`import { db, auth } from './firebase'`) instead of each initializing Firebase separately.

File uploads now go through Cloudinary instead of Firebase Storage. Create `src/uploadImage.js`:

```
const CLOUD_NAME = "your-cloud-name"; // from Step 5
const UPLOAD_PRESET = "trusthome_uploads"; // from Step 5

export async function uploadToCloudinary(file) {
  const formData = new FormData();
  formData.append("file", file);
  formData.append("upload_preset", UPLOAD_PRESET);

  const response = await fetch(
    `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
    { method: "POST", body: formData }
  );

  const data = await response.json();
  return data.secure_url; // save this URL into the Firestore document
}
```

Usage in a listing/upload form is the same shape as before, just pointed at Cloudinary instead of Firebase Storage:

```
const photoUrl = await uploadToCloudinary(selectedFile);
await addDoc(collection(db, "listings"), {
  // ...other fields
  photoUrls: [photoUrl],
});
```

Same pattern applies to the ID/ownership document upload in the listing verification flow — upload to Cloudinary, store the returned URL in `verificationDocUrl`.

Step 9 — Wire Up the Python Scraper

Already handled in the updated `bank_scraper` project — just make sure `serviceAccountKey.json` from Step 7 is in that folder, then:

```
cd bank_scraper
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
python scraper.py
```

Step 10 — Set Up GitHub Actions Secret (for the scheduled scraper)

Since `serviceAccountKey.json` can never be committed, GitHub Actions needs it as an encrypted secret instead:

1. In your GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. Name it `FIREBASE_SERVICE_ACCOUNT`, paste the entire contents of `serviceAccountKey.json` as the value.
3. In `.github/workflows/scrape.yml`, add a step before “Run scraper” that writes the secret back out to a file:

```
- name: Write service account key
  run: echo '${{ secrets.FIREBASE_SERVICE_ACCOUNT }}' > serviceAccountKey.json
```

Step 11 — Lock Down Security Rules (before your defense, not on day one)

Test mode leaves your database wide open — fine for the first week or two of building, but **must be fixed before demo day**. Starting point for Firestore Rules (Build → Firestore Database → Rules tab):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isSignedIn() {
      return request.auth != null;
    }
    function isOwner(userId) {
      return request.auth.uid == userId;
    }

    match /users/{userId} {
      allow read: if true;
      allow write: if isOwner(userId);
    }

    match /listings/{listingId} {
      allow read: if resource.data.verificationStatus == "verified" || isOwner(resource.data.ownerId);
      allow create: if isSignedIn();
      allow update, delete: if isOwner(resource.data.ownerId);
    }

    match /bookings/{bookingId} {
      allow read: if isSignedIn();
      allow create: if isSignedIn();
      allow update: if isSignedIn();
    }

    match /bankProperties/{docId} {
      allow read: if true;
      allow write: if false; // only the scraper (admin SDK) can write here
    }
  }
}
```

Treat this as a starting draft, not a finished ruleset — add proper admin-only checks for the review/dispute queues once the `users` role field is in place, and test each rule against the actual access patterns your app needs before your defense.

Step 12 — Set Up Hosting (do this once, deploy repeatedly)

```
npm install -g firebase-tools
firebase login
# select your project when prompted; build folder is "build" or "dist"
# depending on your React setup (Vite vs Create React App)
firebase init hosting
npm run build
firebase deploy
```

This gives you a live URL (`trusthome-ph.web.app`) you can share with the panel or test on an actual phone — worth doing early, not just before the demo, so everyone can check the deployed version throughout the month.

Quick Reference — Who Needs What

Person	Needs
--------	-------

Everyone (React work)	The <code>firebaseConfig</code> object from Step 6, pasted into their local <code>src/firebase.js</code> — plus the Cloudinary <code>CLOUD_NAME</code> and upload preset name from Step 5
Solpico (scraper)	<code>serviceAccountKey.json</code> from Step 7, placed in the <code>bank_scraper</code> folder
Whoever sets up GitHub Actions	The same key's contents, added as a GitHub secret (Step 10)
Andresa (Scrum Master)	Owner access to the Firebase project, plus admin access to the Cloudinary account, to manage both team member lists

Once Steps 1–9 are done, every module in Week 1 of the Versioning Plan can start immediately — there's no other blocker.